

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

NORMALIZACIÓN AVANZADA Y ELIMINACIÓN DE REDUNDANCIA

En el diseño lógico de bases de datos, la normalización es una técnica esencial que permite estructurar las tablas de manera eficiente, reduciendo la redundancia de datos y evitando anomalías de actualización, inserción y eliminación.

Si bien en los niveles básicos (1NF, 2NF y 3NF) se abordan los aspectos más comunes de la organización de datos, y de las que haremos un breve recordatorio junto con ejemplos que te permitirán entenderlas sencillamente, en sistemas más complejos o con requerimientos más estrictos se hace necesario avanzar hacia formas más profundas de normalización, conocidas como formas normales avanzadas.

IMPORTANTE

En este material veremos sintéticamente las formas básicas y avanzadas, la profundización se encuentra en el material y bibliografía de la cátedra.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

PRIMERA FORMA NORMAL (1NF)

Una tabla está en 1NF si todos los atributos contienen valores atómicos indivisibles, no multivaluados ni repetidos y cada campo contiene un solo valor por fila.

La 1NF pretende eliminar los grupos repetitivos o listas de valores en una misma columna asegurando que cada campo tenga uno solo.

EJEMPLO QUE NO CUMPLE CON LA PRIMERA FORMA NORMAL (1NF)

El problema es que la columna **Cursos** de la tabla **Cursante** tiene valores múltiples.

Estudiante		
ID_Alumno	Nombre	Cursos
1	Ana Pérez	Matemática, Física, Ciencias Naturales, Ciencias Sociales
2	Juan Gómez	Programación

Solucionamos el problema y pasamos a 1NF la columna **Curso** de la tabla **Cursante**

Estudiante		
ID_Alumno	Nombre	Curso
1	Ana Pérez	Matemática
1	Ana Pérez	Física
2	Juan Gómez	Programación

¡IMPORTANTE!

Quizás no lo hayas notado, pero en el ejemplo de la tabla **Estudiante** el nombre de la columna normalizada cambió de **Cursos** a **Curso** ¿Esto es posible o afecta en algo a nuestra colección de datos?, en general, no es obligatorio cambiar el nombre de una columna cuando se normaliza una tabla, solo se hace si es útil para la claridad, coherencia o para evitar confusión en el nuevo modelo.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

SEGUNDA FORMA NORMAL (2NF)

Para que una tabla se encuentre en la 2NF, debe estar en 1NF, es decir que todos sus datos sean atómicos, sin listas ni campos repetidos, y cuando todas sus columnas (atributos) no clave depende completamente de la clave primaria, ya que no existen dependencias parciales en claves compuestas.

La 2NF busca eliminar dependencias parciales en claves compuestas.

ID_Alumno	ID_Curso	Nombre_Alumno
A1	Ana	Matemática
A2	Bruno	Bruno



Alumnos
ID_Alumno
A1
A2

Inscripciones	
ID_Alumno	Curso
A1	C1

Cursos	
ID_Curso	Curso
A1	C1

SINTESIS	
TÉRMINO	SIGNIFICADO
CLAVE SIMPLE	Una sola columna identifica la fila.
CLAVE COMPUESTA	Se necesitan dos o más columnas juntas para identificar una fila de forma única.

¡IMPORTANTE!

Una **dependencia parcial** existe solo cuando la clave primaria está compuesta, es decir, formada por más de una columna.

La **clave compuesta** es una clave primaria formada por dos o más columnas (atributos) que, juntas, identifican de forma única cada fila en una tabla.

¿Por qué usar más de una columna? Porque ninguna de las columnas por separado es suficiente para identificar una fila de manera única.

Un atributo depende parcialmente de la clave cuando solo necesita una parte de esa clave para obtener su valor.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

EJEMPLO QUE NO CUMPLE CON LA SEGUNDA FORMA NORMAL (2NF)

La clave primaria aquí es (ID_Alumno, ID_Curso), porque un alumno puede estar en varios cursos, y un curso tiene varios alumnos.

El problema lo vemos

Nombre_Alumno depende solo de ID_Alumno →  **dependencia parcial**

Nombre_Curso depende solo de ID_Curso →  **dependencia parcial**

LA TABLA NO SE ENCUENTRA EN 2NF

ID_Alumno	ID_Curso	Nombre_Alumno	Nombre_Curso
A1	C1	Ana	Matemática
A2	C1	Bruno	Matemática

PASAMOS A 2NF

Nos damos cuenta de que nuestro ejemplo no está en 2NF observando que **Nombre_Alumno** solo necesita **ID_Alumno**.

Para que se convierta en 2NF se requiere que cada dato dependa de su clave completa, y no de una parte, para ello separamos la tabla original del ejemplo en tres tablas más pequeñas, **Alumnos**, **Cursos** e **Inscripciones**.

Alumnos	
ID_Alumno	Nombre_Alumno
A1	Ana
A2	Bruno

Cursos	
ID_Curso	Nombre_Curso
C1	Matemática

Inscripciones	
ID_Alumno	ID_Curso
A1	C1
A2	C1

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

TERCERA FORMA NORMAL (2NF)

La tabla en 3NF requiere estar en 2NF y no tener dependencias transitivas o indirectas, es decir, que los atributos no clave no dependan de otros atributos no clave.

AMPLIANDO SABERES

Una **dependencia transitiva** o **indirecta**, ocurre cuando *un dato depende de otro dato que no es clave primaria, pero ese otro dato sí depende de la clave primaria*. En otras palabras, **A** depende de **B** y **B** depende de la clave primaria, entonces **A** depende también de la clave primaria, pero de forma indirecta (transitiva).

Entonces, llamamos dependencia transitiva o indirecta cuando en una tabla un dato depende de otro dato que a su vez depende de la clave primaria, lo cual debemos evitarlo para no repetir datos y que las actualizaciones sean más fáciles de realizar.

EJEMPLO QUE NO CUMPLE CON LA TERCERA FORMA NORMAL (2NF)

Donde la clave primaria es **DNI** y **Nombre** depende directamente del **DNI**, lo cual es correcto en los criterios de normalización, pero **Nombre_Departamento** no depende directamente del **DNI**, porque depende de **ID_Departamento**, que sí depende del **DNI**, entonces, **Nombre_Departamento** depende de forma transitiva del **DNI**, aquí es donde surge el problema, porque estamos repitiendo datos innecesariamente, por ejemplo, "**Contabilidad**" cada vez que aparece el **ID "10"** y si **Nombre_Departamento** cambia, tendremos que actualizarlo en muchos lugares, lo cual no es un procedimiento seguro, cómodo y confiable.

LA TABLA NO SE ENCUENTRA EN 3NF

DNI	Nombre	ID_Departamento	Nombre_Departamento
41345678	Ana	10	Contabilidad
43426789	Bruno	20	Recursos Humanos

PASAMOS A 3NF

Para lograrlo separamos la tabla original en dos tablas y de esta forma eliminamos las dependencias transitivas, cumpliendo con la norma.

Empleados		
DNI	Nombre	ID_Departamento
12345678	Ana	10

Departamentos	
ID_Departamento	Nombre_Departamento
10	Contabilidad

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

FORMAS AVANZADA DE NORMALIZACIÓN

En sistemas que manejan gran cantidad de datos heterogéneos o con relaciones complejas, los esquemas en 3NF pueden seguir presentando redundancias sutiles. Estas redundancias no sólo implican mayor uso de espacio, sino que también aumentan la probabilidad de inconsistencias cuando se actualiza la base de datos.

Para tratar de solucionar el problema de las redundancias más allá de la 3NF se pensaron formas normales avanzadas, las que permiten, minimizar la duplicación innecesaria de datos, garantizar la integridad de la información, mejorar la mantenibilidad del sistema, asegurar que las operaciones de inserción, actualización y eliminación se realicen sin efectos no deseados.

Pero atención, un exceso de descomposición y separación en tablas también puede afectar negativamente el rendimiento de ciertas consultas, por lo que, en la práctica, el uso de formas normales avanzadas debe ser cuidadosamente evaluado según cada situación.

Las formas avanzadas de normalización son las de *Boyce-Cood*, de *Dominio y clave* (DKNF), la *Cuarta* (4NF) y la *Quinta* o de *Proyección-Join* (5NF), veremos de que se trata cada una.

¡IMPORTANTE!

En este material abordaremos sintéticamente las formas avanzadas de normalización, la profundización se encuentra en el material y bibliografía de la cátedra.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****FORMA NORMAL DE BOYCE-CODD (BCNF)**

Forma Normal de Boyce-Codd (BCNF) es una versión más estricta de la 3NF que elimina ciertas dependencias funcionales (relaciones entre columnas de una tabla) que aún pueden causar redundancia.

Se aplica cuando, aun cumpliendo con la 3NF, existen ciertas anomalías que pueden aparecer por la presencia de dependencias funcionales no deseadas.

La forma normal de Boyce-Codd (BCNF) se establece solo si en una relación (tabla) toda dependencia funcional $X \rightarrow Y$, entonces X es una clave candidata.

Es decir, si un conjunto de atributos X determina a otros atributos Y , entonces X debe ser clave candidata (algo que por sí solo puede identificar de forma única a una fila), es importante porque si X no es una clave candidata, puede haber redundancia o inconsistencias.

En BCNF, a diferencia de la 3FN, no se permiten excepciones, todas las dependencias deben tener como determinante una clave candidata.

Una excepción en la 3NF nos permite que X no sea clave siempre y cuando Y sea parte de una clave candidata.

EJEMPLO QUE NO CUMPLE CON LA FORMA NORMAL DE BOYCE-CODD (BCNF)

Un caso típico ocurre en una base de datos Universitaria, donde cada profesor da clases siempre en la misma aula, pero un profesor puede enseñar varias materias, entonces cada Profesor trabaja en un Aula, lo cual es una dependencia funcional correcta, pero Profesor no es clave, porque no identifica de forma única cada fila, para eso necesitamos la conjunción (unión) entre Profesor + Materia y aquí tenemos una infracción a la BCNF, aunque es válida como excepción puede cumplir la 3NF.

Profesor	Materia	Aula
Ana	Matemática	A101
Ana	Física	A101
Bruno	Historia	B202

TRANSFORMANDO LA TABLA A LA FORMA NORMAL DE BOYCE-CODD (BCNF)

Para solucionar este problema se divide la tabla original para eliminar la dependencia que causa la infracción:

Tabla 1: aulas por profesor	
Profesor	Aula
Ana	A101
Bruno	B202

Tabla 2: materias por profesor	
Profesor	Materia
Ana	Matemática
Ana	Física
Bruno	Historia

BASE DE DATOS II
CONTENIDOS Y ACTIVIDADES**PASANDO EN LIMPIO**

CARACTERÍSTICA	FORMA NORMAL DE BOYCE-CODD (BCNF)
Basada en	Tercera Forma Normal (3NF).
Condición clave	Toda dependencia funcional debe tener una clave como lado izquierdo ($X \rightarrow Y$, X es clave).
Objetivo	Eliminar todas las dependencias funcionales anómalas.
Se aplica	Cuando 3NF no es suficiente para evitar redundancias.

AMPLIANDO SABERES

Una **dependencia funcional** es una relación entre columnas (atributos) de una tabla en una base de datos.

Una **clave candidata** es un conjunto de una o más columnas que puede identificar de forma única cada fila de la tabla. A veces hay más de una posibilidad, por eso se llaman candidatas, solo una se elige cómo clave primaria, pero todas son válidas para identificar filas.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

CUARTA FORMA NORMAL (4NF)

Aborda la presencia de dependencias multivaluadas, que pueden provocar combinaciones redundantes de datos.

Decimos que estamos en 4NF si nuestra tabla se encuentra normalizada en Boyce-Codd (BCNF) y no contiene dependencias multivaluadas innecesarias o no triviales.

EJEMPLO QUE NO CUMPLE CON LA 4TA FORMA NORMAL (4NF)

Observemos la siguiente tabla con el caso de Ana.

Estudiante	Idioma	Deporte
Ana	Inglés	Fútbol
Ana	Francés	Fútbol
Ana	Inglés	Vóley
Ana	Francés	Vóley

¿Qué sucede aquí? Ana habla dos idiomas (inglés y francés), y practica dos deportes (fútbol y vóley), pero los idiomas y los deportes no están relacionados entre sí, lo que produce que se combinen todos los idiomas con todos los deportes, creando redundancia innecesaria, esta repetición innecesaria es una dependencia multivaluada:

Estudiante →→ Idioma

Estudiante →→ Deporte

AMPLIANDO SABERES

El símbolo →→ representa una dependencia multivaluada.

TRANSFORMANDO LA TABLA A LA 4TA FORMA NORMAL (4NF)

Para solucionar la dependencia funcional se debe realizar una descomposición dividiendo la tabla original en dos tablas separadas, una para cada conjunto multivaluado, de esta forma se evita la repetición innecesaria de combinaciones, reduciendo la redundancia y mejorando la consistencia de los datos.

Estudiante	Idioma
Ana	Inglés
Ana	Francés

Estudiante	Deporte
Ana	Fútbol
Ana	Vóley

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

PASANDO EN LIMPIO

CARACTERÍSTICA	4TA FORMA NORMAL (FNF)
Requisitos previos	La tabla debe estar en BCNF.
Qué elimina	Dependencias multivaluadas.
Símbolo clave	$X \twoheadrightarrow Y$ (X determina múltiples valores independientes de Y).
Objetivo principal	Eliminar combinaciones innecesarias que provocan redundancia.
Acción recomendada	Dividir la tabla en dos o más relaciones sin redundancias.

AMPLIANDO SABERES

Una **dependencia multivalorada** o **multivaluadas** se da cuando para una sola fila, un valor de un atributo está relacionado con múltiples valores de otro atributo, de forma independiente de otros atributos.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

QUINTA FORMA NORMAL (5NF) O DE PROYECCIÓN-JOIN

La 5NF es el nivel más alto de normalización y se aplica para eliminar redundancias muy complejas que pueden aparecer al reconstruir una tabla a partir de varias relaciones mediante joins.

Resuelve problemas de dependencias de unión, que pueden aparecer cuando los datos están descompuestos en varias relaciones y su recomposición introduce información **falsa**.

La 5NF elimina redundancias que surgen cuando una relación compleja entre tres o más columnas no puede dividirse sin perder información o sin generar duplicados al unirlos.

Una tabla está en 5NF si cumple con la 4NF y no existen uniones (joins) que generen filas adicionales o redundantes al descomponer y luego volver a unir la tabla, esto se relaciona con lo que se llama dependencias de join o join-dependencies.

El uso de la 5NF no es común en el día a día, pero muy útil en sistemas empresariales grandes o en sistemas complejos con múltiples relaciones entre varias entidades, por ejemplo: proveedores, productos, clientes, etc.

EJEMPLO QUE NO CUMPLE CON LA 5TA FORMA NORMAL (5NF)

Observemos la siguiente tabla donde el **Proveedor "P1"** ofrece a sus **Clientes "C1"** y **"C2"** el **Producto "A"** y el **Proveedor "P2"** ofrece a sus **Clientes "C1"** y **"C2"** el **Producto "B"**.

Ahora bien, si cada **Proveedor** puede vender ciertos **Productos**, entonces cada **Cliente** puede comprar ciertos **Productos** y cada **Proveedor** puede tener relaciones con ciertos **Clientes**.

Proveedor	Producto	Cliente
P1	A	C1
P1	A	C2
P2	B	C1
P2	B	C2

¿Qué problema puede aparecer? Si intentamos descomponer esta tabla en otras tres tablas: Proveedor–Producto, Producto–Cliente y Proveedor–Cliente y luego intentamos reconstruir la original con joins, podríamos generar combinaciones erróneas o redundantes, lo que evitamos utilizando la 5NF al asegurar que una tabla que fue descompuesta se puede volver a unir correctamente sin generar combinaciones incorrectas o datos redundantes.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES**

Veamos como quedaría la descomposición en las tres tablas: Proveedor–Producto, Producto–Cliente y Proveedor–Cliente.

Proveedor	Producto
P1	A
P2	B

Producto	Cliente
A	C1
A	C2
B	C1
B	C2

Proveedor	Cliente
P1	C1
P1	C2
P2	C1
P2	C2

Ahora vamos a reconstruir la tabla original utilizando las tres tablas con **JOINS**

```
SELECT
    pc.proveedor,
    pp.producto,
    pc.cliente
FROM
    proveedor_cliente pc
JOIN
    proveedor_producto pp ON pc.proveedor = pp.proveedor
JOIN
    producto_cliente pcl ON pp.producto = pcl.producto AND pc.cliente = pcl.cliente;
```

Observemos el resultado

Proveedor	Producto	Cliente
P1	A	C1
P1	A	C2
P1	B	C1
P1	B	C2
P2	B	C1
P2	B	C2
P2	A	C1
P2	A	C2

¿Cuál es el problema aquí? Observamos que en nuestra nueva tabla aparecen combinaciones incorrectas con registros que no estaban en la tabla original, donde el proveedor P1 nunca ofreció el producto B, pero sí aparece en la tabla reconstruida.

Esto se debe a que la consulta con los JOINS puede generar filas que no existían en la tabla original si hay relaciones combinadas que nunca ocurrieron en conjunto. En tablas pequeñas no supondría un grave problema, ya que nos damos cuenta con la simple observación, pero en una base de datos con muchos registros, es algo que debemos tener en cuenta.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

FORMA NORMAL DE DOMINIO Y CLAVE (DKNF)

Busca que todas las restricciones estén expresadas mediante dominios y claves, eliminando cualquier anomalía residual posibles en una tabla, asegurando que todas las restricciones estén representadas únicamente mediante claves y dominios, es la forma más estricta y completa de normalización en bases de datos.

Una tabla está en **Forma Normal de Dominio y Clave** (*Domain Key Normal Form* o **DKNF**) si todas las restricciones sobre los datos pueden expresarse únicamente como restricciones de dominio (valores válidos para un atributo) y restricciones de clave (valores únicos para identificar filas). En **DKNF** no hay restricciones adicionales necesarias para preservar la integridad de los datos.

La **DKNF** evita cualquier tipo de anomalía de inserción, de actualización o de eliminación, esto se logra asegurando que todas las condiciones necesarias para que los datos sean válidos estén incluidas como reglas intrínsecas de la tabla.

Una **Forma Normal de Domino y Clave (DKNF)** raramente se utiliza en bases de datos reales completas, porque muchos sistemas manejan reglas de negocio fuera de la base de datos por ejemplo en sus aplicaciones, pero es ideal como modelo teórico, para entender el límite superior de la normalización.

EJEMPLO QUE NO CUMPLE CON LA FORMA NORMAL DE DOMINIO Y CLAVE (DKNF)

Para graficar la forma DKNF seguiremos con el ejemplo de la Universidad, donde se tiene definido en la tabla de **Personal** que si el **tipo_empleado** es "**Docente**", el **sueldo** no puede ser menor a \$300.000,00, pero esta no es una restricción de dominio ni de clave, es una regla del rol docente, por lo tanto, esta tabla no está en DKNF.

Personal			
legajo	nombre	tipo_empleado	sueldo
1001	Laura	Docente	500000
1002	Mario	Bedel	120000

PASANDO LA TABLA A LA FORMA NORMAL DE DOMINIO Y CLAVE (DKNF)

Para que la tabla **Personal** esté en DKNF, tendríamos que implementar la regla adicional del rol profesional de docente como una restricción CHECK (*si el SGBD lo permite*), rediseñar la base de datos para que esa lógica se exprese a través de claves o dominios válidos, aplicar esta validación desde la lógica de la aplicación por ejemplo desde un formulario web o crear un trigger que valide el sueldo antes de insertar o actualizar.

Empleado	
tipo	sueldo_minimo
Docente	300000.00
Bedel	120000.00
Servicio	100000.00

Empleados			
legajo	nombre	tipo_empleado	sueldo
1001	Laura	Docente	500000.00
1002	Mario	Bedel	150000.00
1003	Carla	Servicio	105000.00

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

La tabla **Empleado** define el dominio válido para cada tipo de empleado, incluyendo el sueldo mínimo esperado.

La tabla **Empleados** contiene los datos de los empleados, con una clave primaria (legajo) y una clave foránea (tipo_empleado) que referencia a la tabla (Empleado) anterior.

AMPLIANDO SABERES

¿Qué son las restricciones de dominio y clave? En DKNF, todas las demás reglas del negocio (como "si es cliente vip, el descuento debe ser mayor al 10%") se transforman en restricciones de dominio o clave, o se trasladan al nivel de lógica de la aplicación.

Una **restricción de dominio** es una regla que limita los valores permitidos en una columna, define el tipo y el rango de valores válidos.

Una **restricción de clave** es una regla que asegura la unicidad e identificación de los registros (filas) en una tabla.

En DKNF solo se aceptan este tipo de restricciones, si una regla de integridad no puede expresarse como clave o dominio, la tabla no está en DKNF.

PASANDO EN LIMPIO

TIPO DE RESTRICCIÓN	QUÉ CONTROLA	EJEMPLO
Clave	Identidad y unicidad de filas.	Cada cliente tiene un único DNI.
Dominio	Qué valores puede tener un atributo.	La edad debe ser un número positivo.